

Multimodal NLP

UCU NLP Course

What “Modality” Means?

What “Modality” Means?

A **modality** is a channel of information or a way in which data is expressed. Each modality has its own structure, noise, and semantics.

Typical modalities in multimodal NLP:

1. **Text - sequences of discrete tokens**
2. **Images - continuous pixel grids**
3. Audio - waveforms or spectrograms
4. Video - sequences of images + temporal dynamics
5. Speech - audio + linguistic content
6. Screen frames / UI elements - structured visual interfaces
7. Sensor data - numerical time series
8. Actions / events / keystrokes - temporal symbolic streams

Why Multimodality Matters?

Why Multimodality Matters?

1. Language is grounded in perception.
 - a. Words like “chair,” “open,” “blue,” “above” only make sense in context of the physical world.
2. Multimodal signals reduce ambiguity.
 - a. Text alone often misses nuance - images, audio, and interactions anchor meaning.
3. Better generalization and robustness.
 - a. Models that align vision and language tend to be more stable, less hallucination-prone, and more adaptable to new tasks.⁴
4. Emergence of real-world AI assistants.
 - a. To act, understand, or predict, an agent must combine multiple modalities — chat, images, screens, UI elements, audio, etc.

What “Modality” Means?

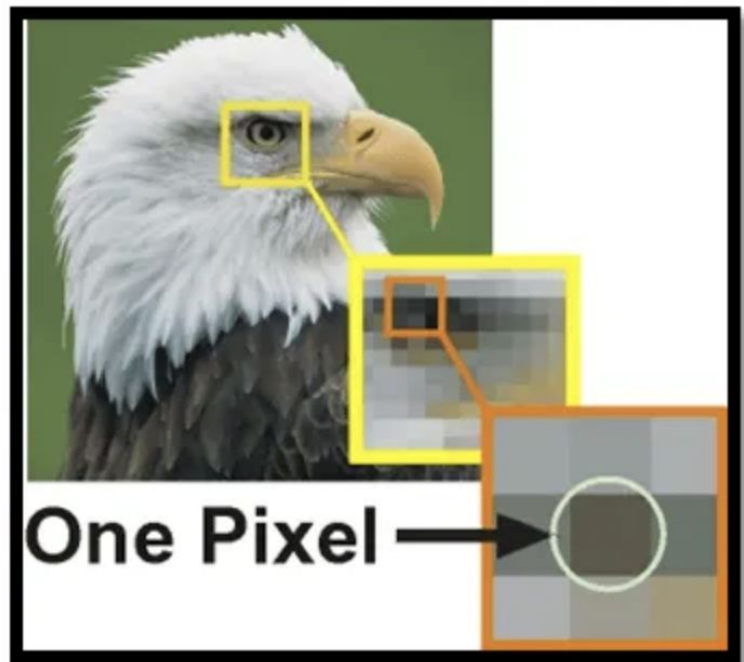
A **modality** is a channel of information or a way in which data is expressed. Each modality has its own structure, noise, and semantics.

Typical modalities in multimodal NLP:

1. Text - sequences of discrete tokens
2. Images - continuous pixel grids
3. Audio - waveforms or spectrograms
4. Video - sequences of images + temporal dynamics
5. Speech - audio + linguistic content
6. Screen frames / UI elements - structured visual interfaces
7. Sensor data - numerical time series
8. Actions / events / keystrokes - temporal symbolic streams

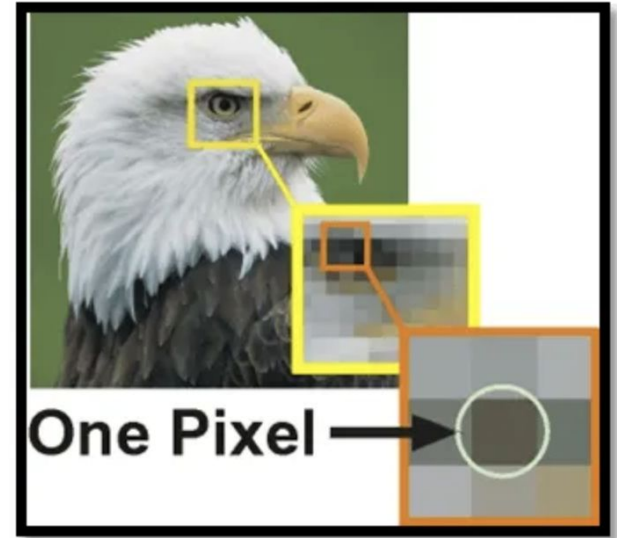
CNN Recap

What is the image?



What is the image?

1. Digital images are essentially grids of tiny units called pixels.
2. Typically, each pixel is composed of three values corresponding to the red, green, and blue (RGB) color channels.
3. Examine the [tool](#).



Main CNN building blocks

- Convolution Operation

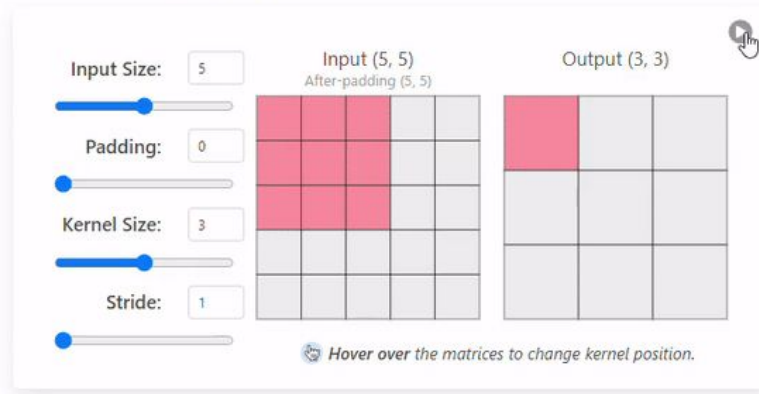
- a. The convolution operation involves multiplying the **kernel** values by the original pixel values of the image and then summing up the results.

Input		Kernel		Output																	
<table border="1"><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table>	0	1	2	3	4	5	6	7	8	*	<table border="1"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table>	0	1	2	3	=	<table border="1"><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table>	19	25	37	43
0	1	2																			
3	4	5																			
6	7	8																			
0	1																				
2	3																				
19	25																				
37	43																				

Main CNN building blocks

- Stride
 - a. The number of pixels by which a kernel moves across the input image.
 - b. Increasing the stride will allow the filter to cover a larger area of the input image, which can be useful for capturing **more global features**.
 - c. Lowering the stride will capture finer and more **local details**.

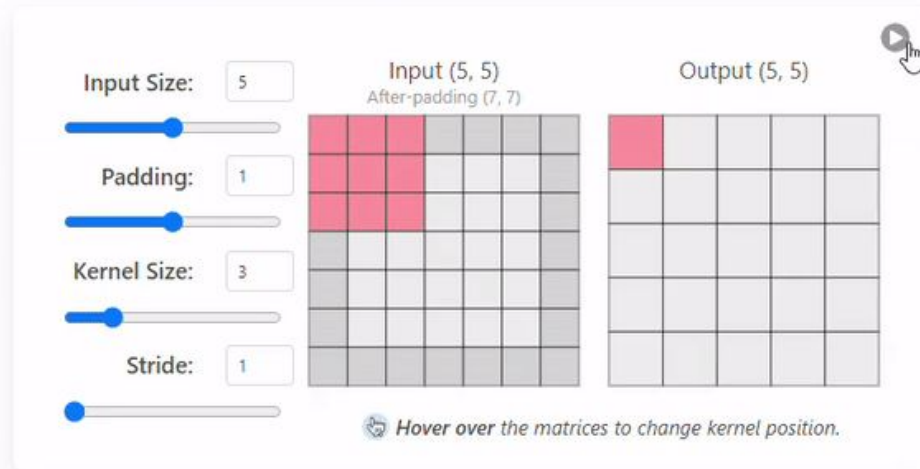
Understanding Hyperparameters



Main CNN building blocks

- Padding
 - a. The purpose of padding is to adjust the spatial size of the output of a convolutional operation and to preserve spatial information at the borders.

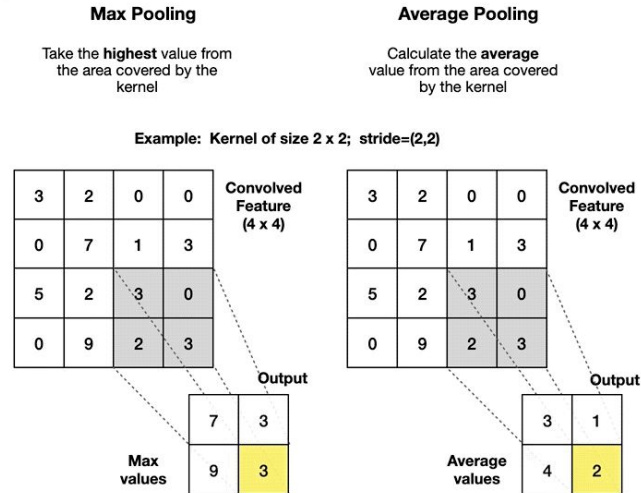
Understanding Hyperparameters



Main CNN building blocks

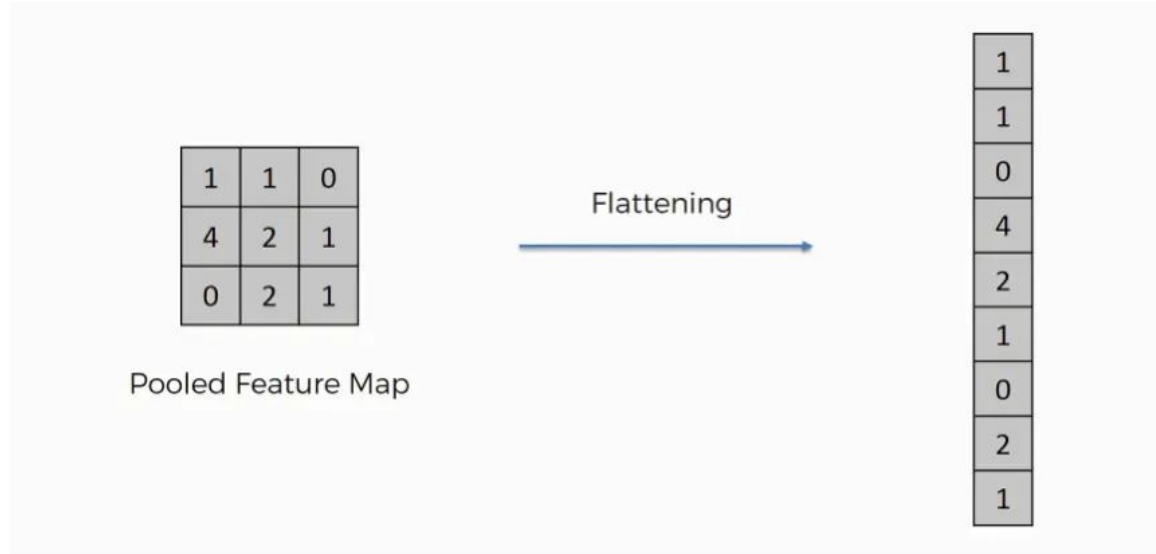
- Pooling

- In pooling, we are not applying any kernel to the input data, we are just simplifying the information with a math operation (Max or Avg).

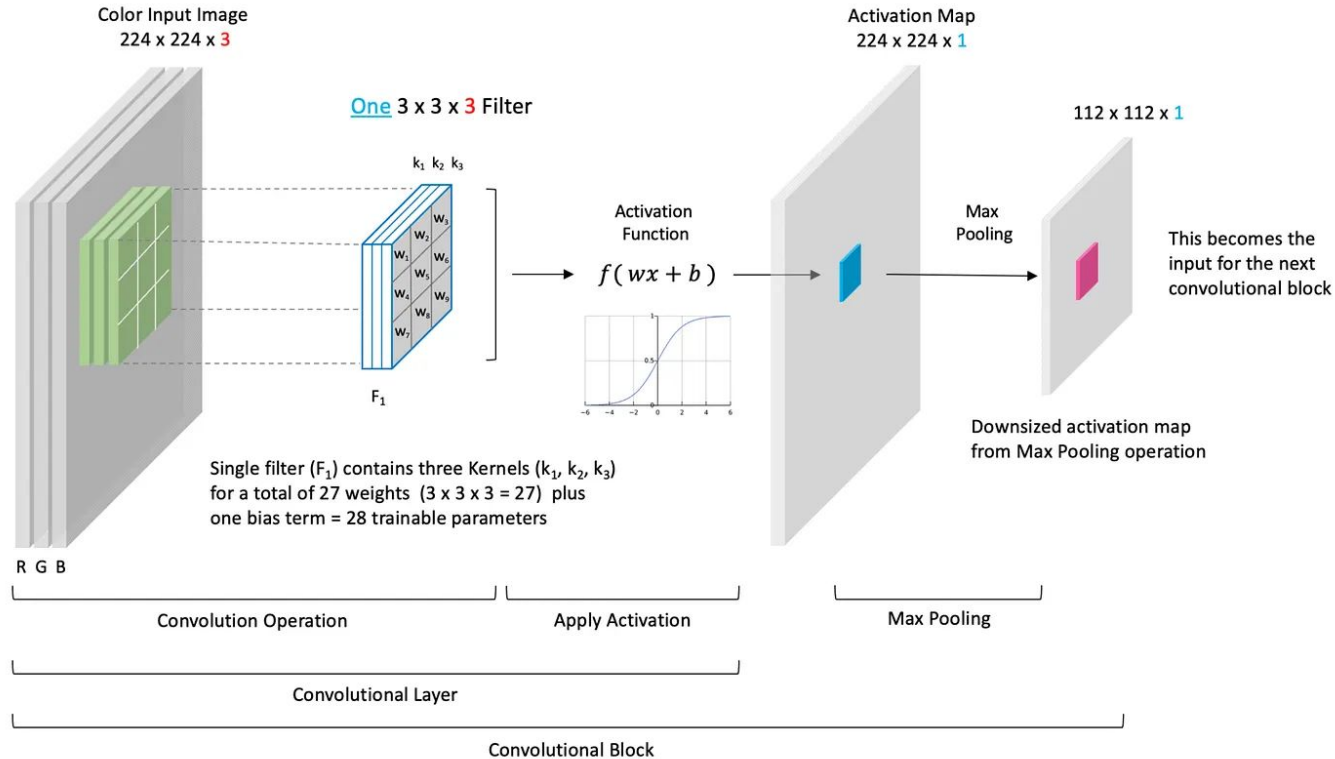


Main CNN building blocks

- Flattening layers

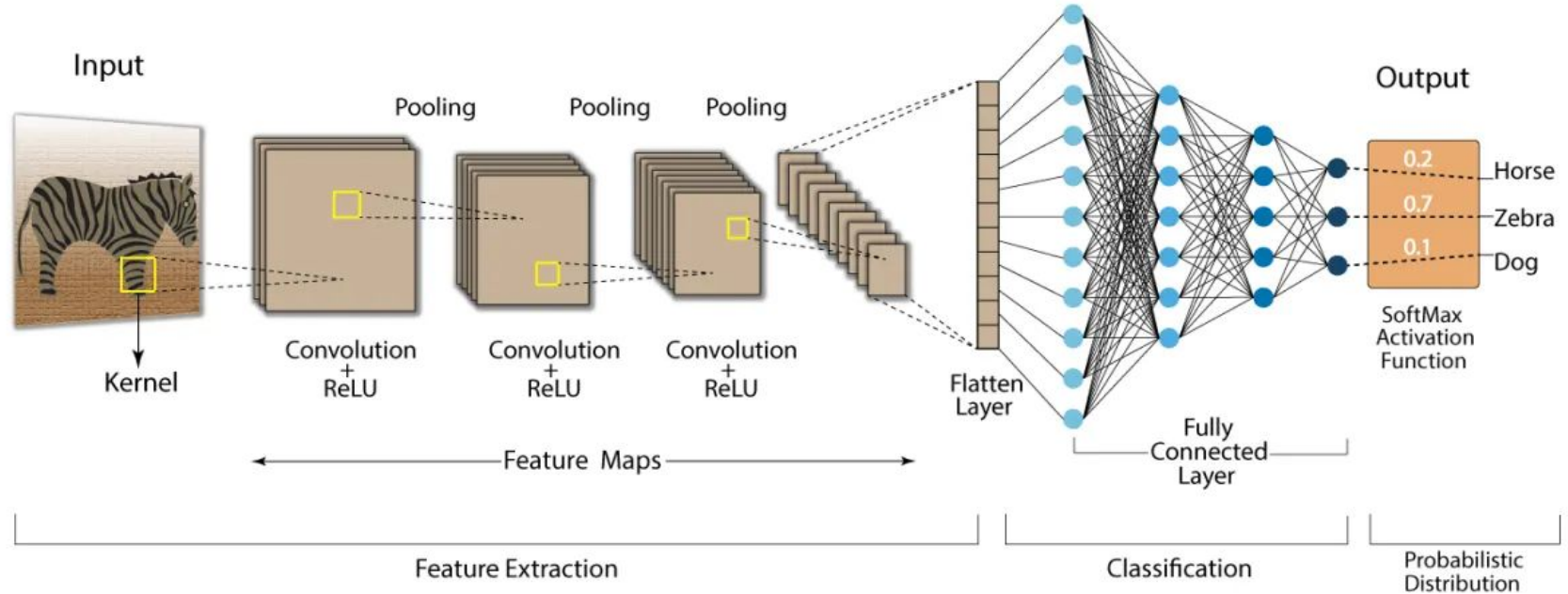


Convolutional block



Convolutional Neural Network

Convolution Neural Network (CNN)



ViT

Visual Image Transformer

What is the high-level idea?

Visual Image Transformer

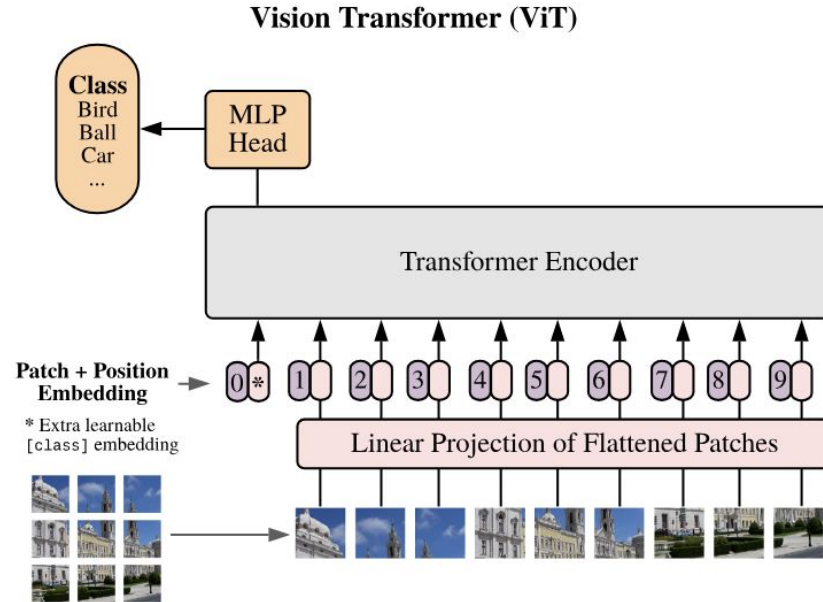
What is the high-level idea?

Replace CNNs with a **plain Transformer encoder** for images by treating an image as a **sequence of patches (tokens)**, and show that with large-scale pretraining, this “Vision Transformer” can **match or beat state-of-the-art CNNs** on image classification.

Visual Image Transformer

How can we utilize image with Transformer?

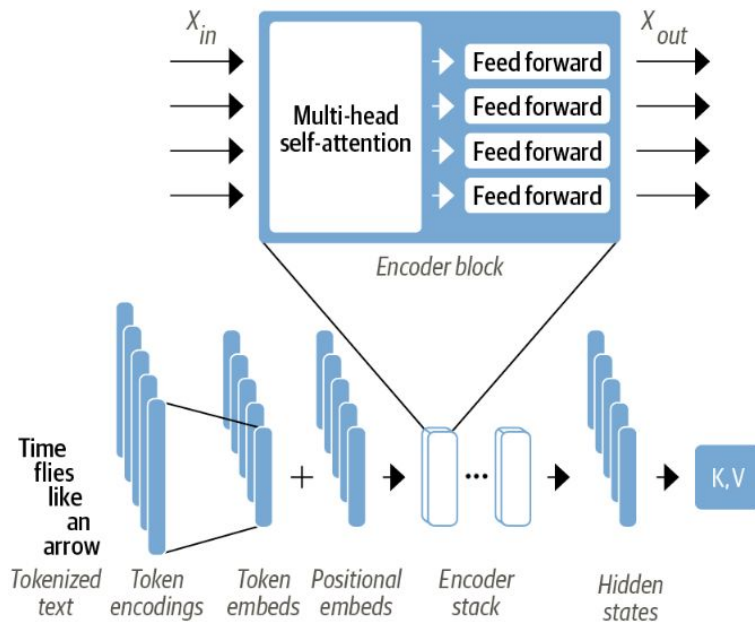
Visual Image Transformer



Visual Image Transformer

1. Image: $32 \times 32 \rightarrow$ patch size $16 \times 16 \rightarrow$ gives 4 patches
2. Each patch has:
 - a. $16 \times 16 \times 3 = 768$ values
3. Project into pixels (each patch) into embedding
4. Add classification token
5. Add positional embedding
6. Feed into Transformer

Transformer Anatomy Recap



Inductive bias vs data scale

CNN has strong inductive bias:

1. Nearby pixels matter more than far pixels → convolution.
2. Images have local edges and textures → small filters.

The model “believes” these rules even before learning anything.

ViT has weak inductive bias:

1. It does not assume images have local patterns.
2. It does not assume spatial structure.
3. It only knows “image = sequence of patches.”

Inductive bias vs data scale

With strong inductive bias (CNN):

- Model learns good features even with small datasets.

With weak inductive bias (ViT):

- Model needs massive data to discover structure on its own.

This makes ViT:

1. More flexible
2. Potentially more powerful

Model variants and patch sizes

Authors reuse the BERT sizing scheme:

1. ViT-Base (ViT-B): 12 layers, $D = 768$, 12 heads, ~86M params.
2. ViT-Large (ViT-L): 24 layers, $D = 1024$, 16 heads, ~307M params.
3. ViT-Huge (ViT-H): 32 layers, $D = 1280$, 16 heads, ~632M params.

Authors vary the patch size:

1. Notation: e.g. ViT-L/16 = Large model with 16×16 patches.
2. Smaller patch size $\rightarrow$ longer sequence $\rightarrow$ higher compute.

Hybrid model (CNN + ViT)

1. First, pass the image through a CNN (e.g. ResNet).
2. Take some intermediate feature map.
3. Treat its spatial positions as “patches,” flatten and project each position into D-dim space.
4. Feed that sequence to the Transformer.

This lets you reuse CNN inductive bias in early layers and use ViT for global reasoning.

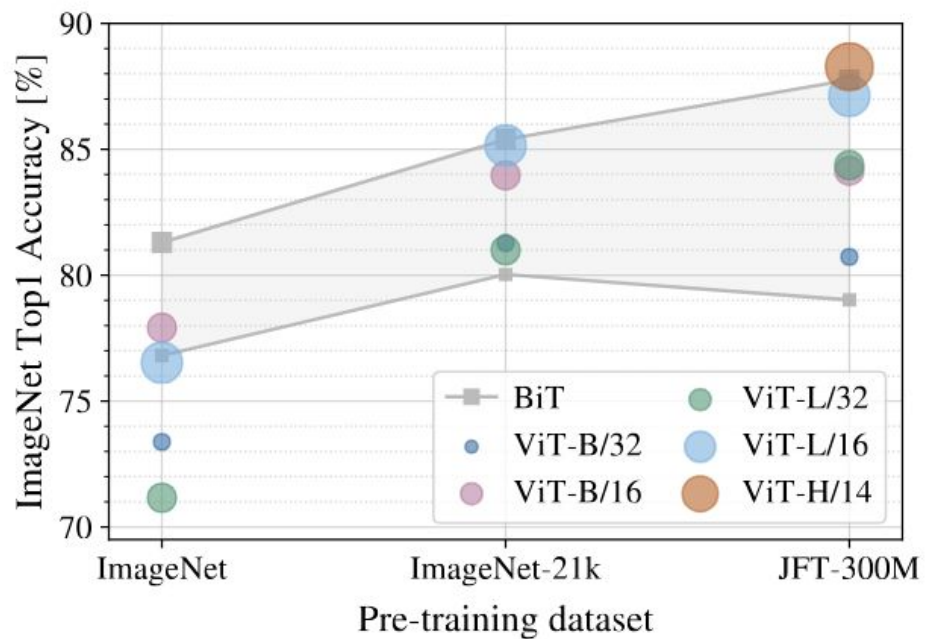
Datasets and pretraining

1. ImageNet-1k: 1.3M images, 1k classes.
2. ImageNet-21k: 14M images, 21k classes.
3. JFT-300M (internal): 303M images, ~18k classes.

Performance

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet ReaL	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

Data vs Model Scaling



CLIP

Contrastive Language–Image Pre-Training

Train a **dual-encoder image-text model** on 400M web image-caption pairs to predict **which text goes with which image**; then use natural language prompts as classifiers for zero-shot vision tasks.

What do you think is the motivation behind CLIP?

Instead of fixed labels like “dog, cat, car” → **use natural language as supervision!**

Data: WebImageText (WIT)

Authors build a large web dataset:

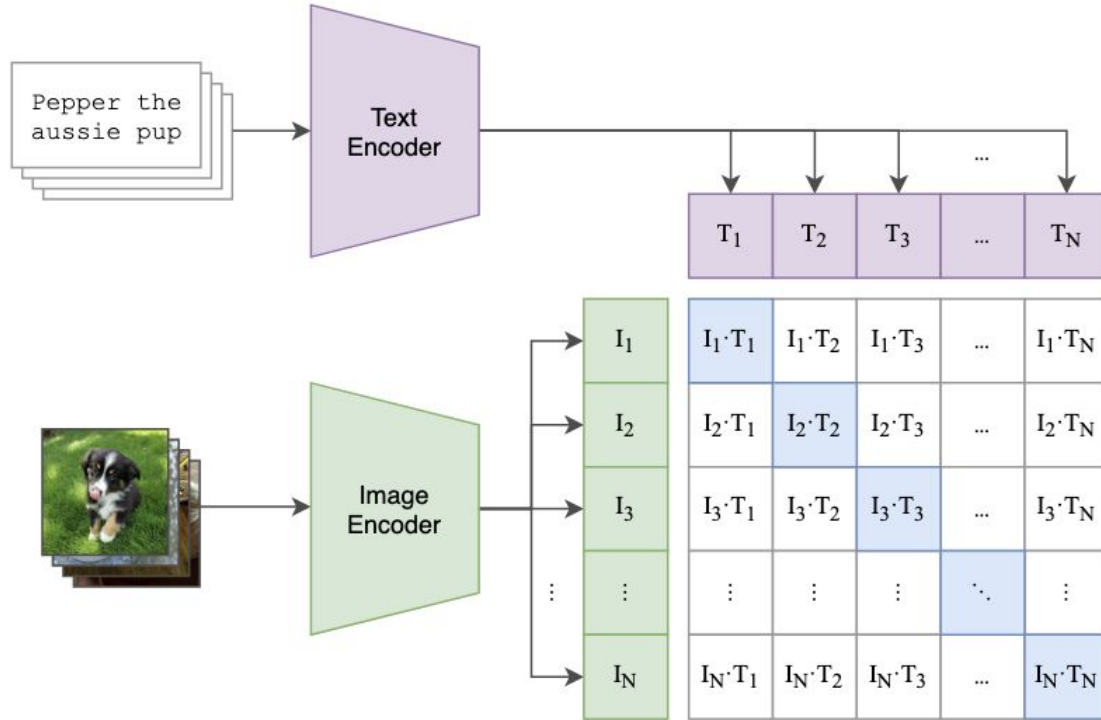
1. Start from a huge list of text queries (words/phrases from Wikipedia, etc.).
2. Use them to search the web and scrape (image, text) pairs.
3. Cap each query at up to 20k image-text pairs to avoid a few queries dominating.

Final dataset: about 400 million (image, text) pairs, called WebImageText (WIT).

Total word count is similar to GPT-2's WebText.

So CLIP is trained on natural alt-text / captions from the web, not curated labels like ImageNet classes.

Architecture + Contractive pre-training



Architecture + Contractive pre-training

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t            - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

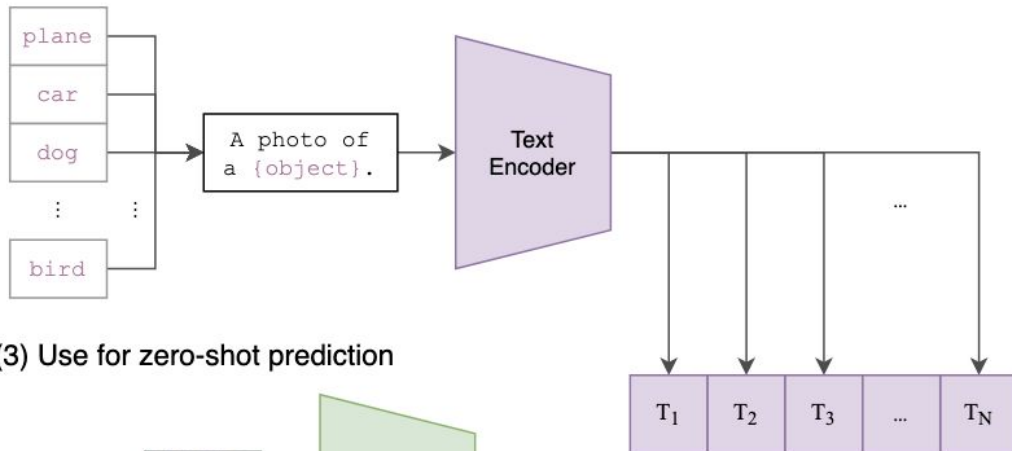
# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

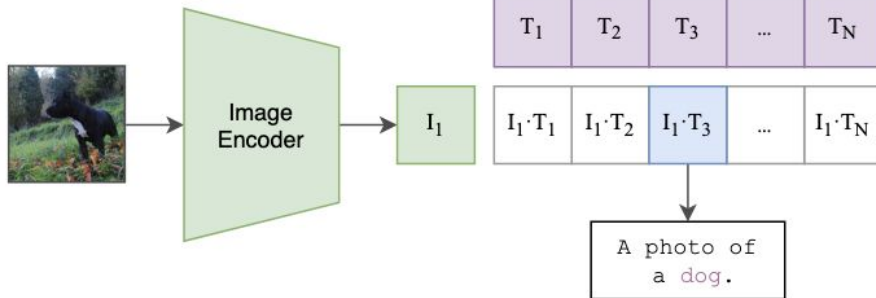
What do you think is the main power of CLIP?

Zero Shot Nature

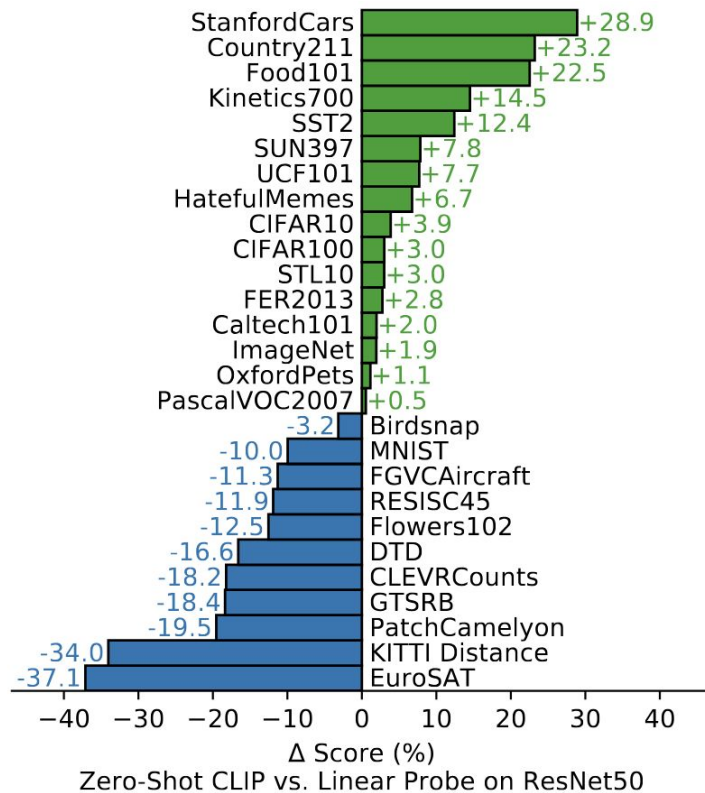
(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



Zero-shot CLIP is competitive with a fully supervised baseline



Zero-shot CLIP is much more robust to distribution shift than standard ImageNet models

	Dataset Examples	ImageNet ResNet101	Zero-Shot CLIP	Δ Score
ImageNet		76.2	76.2	0%
ImageNetV2		64.3	70.1	+5.8%
ImageNet-R		37.7	88.9	+51.2%
ObjectNet		32.6	72.3	+39.7%
ImageNet Sketch		25.2	60.2	+35.0%
ImageNet-A		2.7	77.1	+74.4%

Do you see any problems with CLIP?

Do you see any problems with CLIP?

1. Needs large batches.
2. Tightly couples loss to batch size.
3. Requires full $N \times N$ similarity matrix across devices.

SigLIP

SigLIP

Replace CLIP's softmax contrastive loss with a simple **sigmoid (binary logistic)** loss over image-text pairs, to make language-image pre-training simpler, more scalable, and often better, especially at smaller batch sizes.

SigLIP

1. Instead of softmax over each row/column, we treat each pair as a binary classification
2. Each pair (image, text) gets a similarity score
3. Apply a sigmoid:

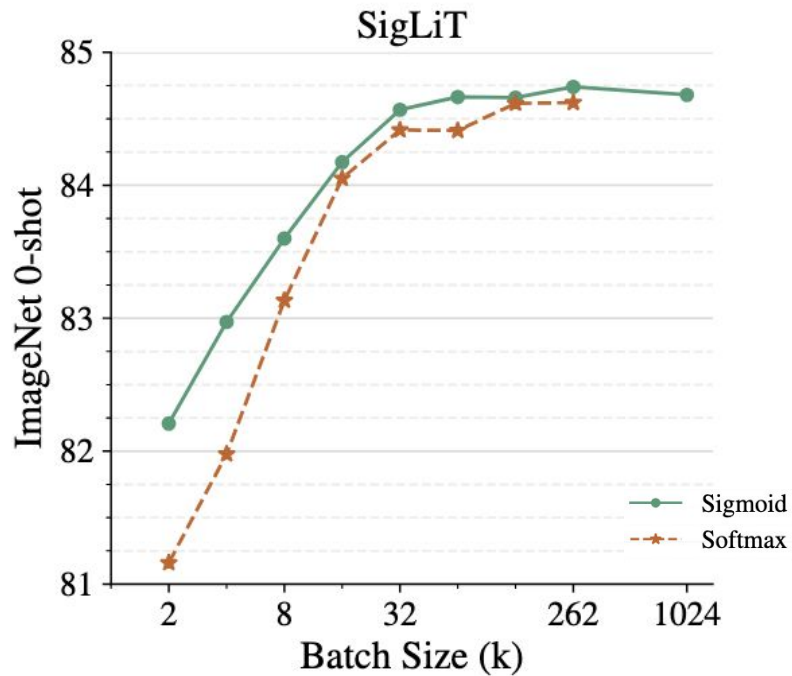
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

4. Use binary cross-entropy (logistic loss) over positives and a set of negatives.

Key differences vs softmax

1. No softmax over the batch dimension.
2. Each pair's contribution is independent; you don't need the full $N \times N$ normalization.

Batch size impact



Performance

Method	Image Encoder		ImageNet-1k				COCO R@1	
	ViT size	# Patches	Validation	v2	Real	ObjectNet	I → T	T → I
CLIP	B	196	68.3	61.9	-	55.3	52.4	33.1
OpenCLIP	B	196	70.2	62.3	-	56.0	59.4	42.3
EVA-CLIP	B	196	74.7	67.0	-	62.3	58.7	42.2
SigLIP	B	196	76.2	69.6	82.8	70.7	64.4	47.2
SigLIP	B	256	76.7	70.0	83.1	71.3	65.1	47.4
SigLIP	B	576	78.6	72.1	84.5	73.8	67.5	49.7
SigLIP	B	1024	79.2	73.0	84.9	74.7	67.6	50.4
CLIP	L	256	75.5	69.0	-	69.9	56.3	36.5
OpenCLIP	L	256	74.0	61.1	-	66.4	62.1	46.1
CLIPA-v2	L	256	79.7	72.8	-	71.1	64.1	46.3
EVA-CLIP	L	256	79.8	72.9	-	75.3	63.7	47.5
SigLIP	L	256	80.5	74.2	85.9	77.9	69.5	51.1
CLIP	L	576	76.6	72.0	-	70.9	57.9	37.1
CLIPA-v2	L	576	80.3	73.5	-	73.1	65.5	47.2
EVA-CLIP	L	576	80.4	73.8	-	78.4	64.1	47.9
SigLIP	L	576	82.1	75.9	87.0	81.0	70.6	52.7
OpenCLIP	G (2B)	256	80.1	73.6	-	73.0	67.3	51.4
CLIPA-v2	H (630M)	576	81.8	75.6	-	77.4	67.2	49.2
EVA-CLIP	E (5B)	256	82.0	75.7	-	79.6	68.8	51.1
SigLIP	SO (400M)	729	83.2	77.2	87.5	82.9	70.2	52.0

Vision Instruction Tuning

LLaVA = Large Language-and-Vision Assistant.

1. Instruction tuning (Alpaca, FLAN, etc.) showed that LLMs become good zero-shot task followers when fine-tuned on synthetic instructions.
2. But there was no equivalent for vision + language.
3. They propose “**visual instruction tuning**”. It consists of:
 - a. Approach to build vision instruction dataset.
 - b. Approach to achieve multimodal (image+text) LLM.

GPT-assisted Visual Instruction Data Generation

Question: How do we turn plain image-caption datasets into rich multimodal instruction-following data using GPT-4?

GPT-assisted Visual Instruction Data Generation

Question: How do we turn plain image-caption datasets into rich multimodal instruction-following data using GPT-4?

- We already have tons of image–text pairs (e.g. CC3M, LAION, COCO).
- But we have almost no data of the form: “Human asks an instruction-style question about an image → Assistant answers in a helpful, detailed way.”
- Getting that kind of multimodal instruction-following data by hand is expensive and ill-defined.

GPT-assisted Visual Instruction Data Generation

Question: How do we turn plain image-caption datasets into rich multimodal instruction-following data using GPT-4?

- We already have tons of image–text pairs (e.g. CC3M, LAION, COCO).
- But we have almost no data of the form: “Human asks an instruction-style question about an image → Assistant answers in a helpful, detailed way.”
- Getting that kind of multimodal instruction-following data by hand is expensive and ill-defined.
- **So they propose:** use text-only GPT-4 / ChatGPT as a teacher to synthesize multimodal instructions and answers from existing image-text datasets.

Trivial Approach to Get The Vision Instruction Dataset

Start with one image-caption pair: X_v , X_c .

The simplest way to convert it into an instruction format:

1. Use GPT-4 **once** to generate a list of “describe this image briefly” questions:
 - a. “Describe the image concisely”,
 - b. “Provide a brief description of the given image”, etc.
2. For each image-caption pair, sample one question, and build a chat-style training example:

Human: X_q X_v <STOP>

Assistant: X_c <STOP>

3. The question is synthetic, but the answer is the original caption.

Trivial Approach to Get The Vision Instruction Dataset

This is very cheap to construct.

But it's shallow:

1. Instructions lack diversity.
2. Answers are just short captions, no long-form explanation or deep reasoning.

Semi-supervised approach (proposed by the authors)

Key idea: encode the image as symbolic text, then ask GPT-4.

At the time, GPT-4 they used is text-only → it can't “see” pixels.

Semi-supervised approach (proposed by the authors)

Key idea: encode the image as symbolic text, then ask GPT-4.
At the time, GPT-4 they used is text-only → it can't “see” pixels.

Context type 1: Captions

A group of people standing outside of a black vehicle with various luggage.

Luggage surrounds a vehicle in an underground parking area

People try to fit all of their luggage in an SUV.

The sport utility vehicle is parked in the public garage, being packed for a trip

Some people with luggage near a van that is transporting it.

Context type 2: Boxes

person: [0.681, 0.242, 0.774, 0.694], backpack: [0.384, 0.696, 0.485, 0.914], suitcase: ...<omitted>



This symbolic scene description = captions + object labels + coordinates.

Three types of GPT-generated instruction-following data

Response type 1: conversation

Question: What type of vehicle is featured in the image?

Answer: The image features a black sport utility vehicle (SUV) ...<omitted>

Response type 2: detailed description

The image is an underground parking area with a black sport utility vehicle (SUV) parked. There are three people in the scene, with one person standing closer to the left side of the vehicle, another person in the middle, and the third person on the right side. They are all working together to pack their luggage into the SUV for a trip. ...<omitted>

Response type 3: complex reasoning

Question: What challenges do these people face?

Answer: In the image, a group of people is standing outside a black SUV in a parking area, surrounded by various pieces of luggage, including suitcases and backpacks. They are facing the challenge of fitting all their luggage into the black SUV. There are multiple suitcases and backpacks to be packed, which suggests that the group has a significant amount of belongings ...<omitted>

Final dataset: LLaVA-Instruct-158K

Using GPT-4/ChatGPT in this way on COCO images + symbolic context, they generate:

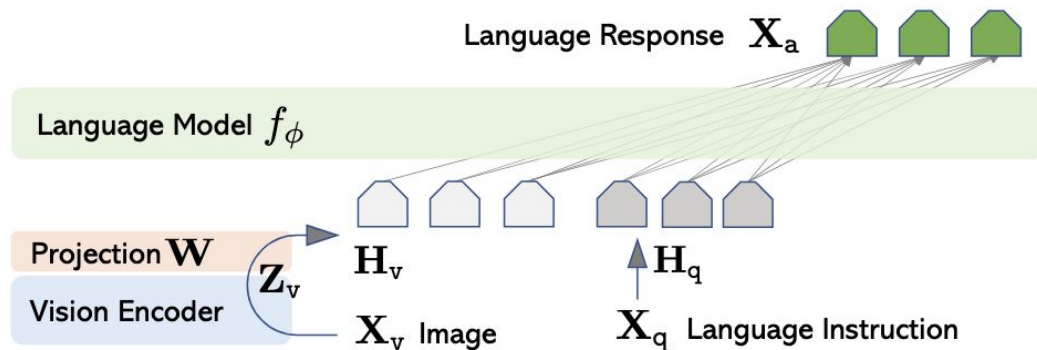
158k total language–image instruction-following samples, consisting of:

1. 58k conversation examples.
2. 23k detailed descriptions.
3. 77k complex reasoning examples.

They note an ablation:

1. Tried ChatGPT vs GPT-4 as the teacher.
2. GPT-4 consistently gave higher-quality data, especially for spatial reasoning, so they prefer GPT-4.

Architecture



$$H_v = W \cdot Z_v, \text{ with } Z_v = g(X_v)$$

1. Vicuna is used as the LLM
2. CLIP ViT-L/14 is used as the visual encoder

Training pipeline (Define the instruction at each turn)

For each image $\mathbf{X}_v$, we generate multi-turn conversation data $(\mathbf{X}_q^1, \mathbf{X}_a^1, \dots, \mathbf{X}_q^T, \mathbf{X}_a^T)$, where T is the total number of turns. We organize them as a sequence, by treating all answers as the assistant's response, and the instruction $\mathbf{X}_{\text{instruct}}^t$ at the t -th turn as:

$$\mathbf{X}_{\text{instruct}}^t = \begin{cases} \text{Randomly choose } [\mathbf{X}_q^1, \mathbf{X}_v] \text{ or } [\mathbf{X}_v, \mathbf{X}_q^1], & \text{the first turn } t = 1 \\ \mathbf{X}_q^t, & \text{the remaining turns } t > 1 \end{cases} \quad (2)$$

Intuition: The image is shared across all turns and is always considered as context, but it's only explicitly inserted as tokens in the first turn.

Training pipeline (Build training sample)

For each image $\mathbf{X}_v$, we generate multi-turn conversation data $(\mathbf{X}_q^1, \mathbf{X}_a^1, \dots, \mathbf{X}_q^T, \mathbf{X}_a^T)$, where T is the total number of turns. We organize them as a sequence, by treating all answers as the assistant's response, and the instruction $\mathbf{X}_{\text{instruct}}^t$ at the t -th turn as:

$$\mathbf{X}_{\text{instruct}}^t = \begin{cases} \text{Randomly choose } [\mathbf{X}_q^1, \mathbf{X}_v] \text{ or } [\mathbf{X}_v, \mathbf{X}_q^1], & \text{the first turn } t = 1 \\ \mathbf{X}_q^t, & \text{the remaining turns } t > 1 \end{cases} \quad (2)$$

Training pipeline (Build training sample)

```
Xsystem-message <STOP>  
Human : Xinstruct1 <STOP> Assistant: Xa1 <STOP>  
Human : Xinstruct2 <STOP> Assistant: Xa2 <STOP> ...
```

The input sequence used to train the model.

1. In practice, the number of turns varies based on the instruction-following data.
2. The model is trained to predict the assistant answers and where to stop,
3. Only green sequence/tokens are used to compute the loss in the auto-regressive model.

Training pipeline (Autoregressive training)

Specifically, for a sequence of length L , we compute the probability of the target answers $\mathbf{X}_a$ by:

$$p(\mathbf{X}_a | \mathbf{X}_v, \mathbf{X}_{\text{instruct}}) = \prod_{i=1}^L p_{\theta}(\mathbf{x}_i | \mathbf{X}_v, \mathbf{X}_{\text{instruct}, < i}, \mathbf{X}_{a, < i}), \quad (3)$$

θ = trainable parameters (depends on which stage we're in)

$\mathbf{X}_{\text{instruct}, < i}$ = all instruction tokens before position i across turns

$\mathbf{X}_{a, < i}$ = all assistant tokens before position i (i.e. previous outputs)

Training pipeline (Autoregressive training)

Specifically, for a sequence of length L , we compute the probability of the target answers $\mathbf{X}_a$ by:

$$p(\mathbf{X}_a | \mathbf{X}_v, \mathbf{X}_{\text{instruct}}) = \prod_{i=1}^L p_{\theta}(\mathbf{x}_i | \mathbf{X}_v, \mathbf{X}_{\text{instruct}, < i}, \mathbf{X}_{a, < i}), \quad (3)$$

θ = trainable parameters (depends on which stage we're in)

$\mathbf{X}_{\text{instruct}, < i}$ = all instruction tokens before position i across turns

$\mathbf{X}_{a, < i}$ = all assistant tokens before position i (i.e. previous outputs)

Which loss should we use?

Architecture

1. Vision encoder

- a. CLIP ViT-L/14 from OpenAI, frozen.
- b. Input: RGB image (224×224).
- c. Output: a sequence of visual embeddings (patch tokens).

2. Language model

- a. Vicuna-7B or Vicuna-13B, an instruction-tuned LLaMA variant (LLaMA pretrained on web/books/code; Vicuna fine-tuned on ShareGPT chats).

Training pipeline: two stages + one downstream task

1. **Stage 1:** Feature alignment - make CLIP features “speak LLaMA”.
2. **Stage 2:** Visual instruction tuning - train on GPT-4 multimodal instructions.
3. Optional task-specific fine-tuning on ScienceQA.

Stage 1 – Feature alignment (CC-595K)

Goal: only learn the projection layer so that visual embeddings look like token embeddings.

Dataset: LLaVA-CC3M-Pretrain-595K: a coverage-balanced subset of Conceptual Captions 3M (CC3M), filtered so that noun phrases are better covered and not overly dominated by frequent ones.

Stage 2 - Visual Instruction Tuning

Goal: fine-tune Vicuna + projection (vision encoder still frozen).

Dataset: LLaVA-Instruct-158K

Stage 3 - Task-specific fine-tuning: ScienceQA

Goal: to show the model isn't just “demo chatty”, authors fine-tune on ScienceQA: a 21k-question multimodal multiple-choice science benchmark.

Evaluation

Scoring protocol: GPT-4 as judge

LLaVA-Bench (COCO):

- 30 unseen COCO images $\times$ 3 instruction types $\rightarrow$ 90 examples.
- 85.1% of GPT-4's quality.

LLaVA-Bench (In-the-Wild):

- 24 natural images (photos, memes, paintings, sketches, UI screenshots, etc.)
- 60 questions in total, with detailed human reference descriptions.
- 67.3% of GPT-4's quality.

ScienceQA

- Accuracy: 90.92%.

Q/A